

DATA STRUCTURES & ALGORITHMS

ONE NIGHT MEGA PREPARATION GUIDE

CSC211 · Spring 2026 · COMSATS Lahore · BS(SE)

■ Stack ■ Queue ■ Linked List ■ BST ■ AVL ■ Heap

Includes: Quiz 2 Pattern Questions · Mid-Term Pattern · Full Concept Traces

Table of Contents

#	Section	Key Topics
1	STACK	Push/Pop, Binary↔Decimal, Infix→Postfix, Postfix Eval, Balanced Brackets
2	QUEUE	Linear Queue, Circular Queue, Double-Ended Queue (Deque)
3	LINKED LIST	Singly LL, Reverse LL (Quiz Q), Remove Duplicates (Quiz Q), Circular LL, Doubly LL, Merge, Rotate
4	BINARY SEARCH TREE	Insert, Delete (3 cases), Traversals, BST from Past Paper
5	AVL TREE	Balance Factor, Height, LL/RR/LR/RL Rotations, Full Trace
6	HEAP TREE	Max/Min Heap, Insert, Delete, Heap Sort, Priority Queue
7	PRACTICE EXAM	Full Mock Paper — Quiz 2 + Mid-Term Pattern

SECTION 1: STACK

1.1 What is a Stack?

LIFO Principle

Last In, First Out — the last element pushed is the first to be popped. Think of a stack of books: you always pick from the top.

push(x)	Add x to top. If size==MAX → Overflow error. arr[top++]=x
pop()	Remove & return top. If top==-1 → Underflow error. return arr[--top]
peek()	Return top element without removing: arr[top-1]
isEmpty()	Return true if top == -1 (or size == 0)
isFull()	Return true if top == MAX-1 (or size == MAX)

1.2 Decimal → Binary using Stack

Divide the decimal number by 2 repeatedly. Push each remainder. Pop all remainders — this gives the binary number MSB first.

Step	Decimal	÷2	Remainder	Stack
1	25	12	1	[1]
2	12	6	0	[1, 0]
3	6	3	0	[1, 0, 0]
4	3	1	1	[1, 0, 0, 1]
5	1	0	1	[1, 0, 0, 1, 1]
Pop All	—	—	—	11001

✓ Answer: 25₁₀ = 11001₂ | Binary→Decimal: pop digits, multiply by positional powers of 2

1.3 Infix → Postfix Conversion (Stack Algorithm)

Operator Precedence: ^ (3) > * / (2) > + - (1)

Operand (A-Z, 0-9)	Output it directly
'('	Push onto stack
)'	Pop and output until '(' is found; discard both '(' and ')'
Operator	Pop & output all operators with >= precedence from stack, then push current
End of input	Pop all remaining operators from stack to output

FULL TRACE — Past Paper Question: A+((B-C)*(D-E)+F)/G*(H-J)

Token	Stack (bottom→top)	Output	Action
A	[]	A	Operand→output
+	[+]	A	Push +
(	[+, (]	A	Push (
(	[+, (, (]	A	Push (
B	[+, (, (]	A B	Operand→output

-	[+, (, (, -]	A B	Push -
C	[+, (, (, -]	A B C	Operand→output
)	[+, (]	A B C -	Pop until (→ output -
*	[+, (, *]	A B C -	* > nothing left before (, push *
(	[+, (, *, (]	A B C -	Push (
D	[+, (, *, (]	A B C - D	Operand→output
-	[+, (, *, (, -]	A B C - D	Push -
E	[+, (, *, (, -]	A B C - D E	Operand→output
)	[+, (, *]	A B C - D E -	Pop until (→ output -
+	[+, (, +]	A B C - D E - *	+ <= * → pop *, push +
F	[+, (, +]	A B C - D E - * F	Operand→output
)	[+]	A B C - D E - * F +	Pop until (→ output +
/	[+, /]	A B C - D E - * F +	/ > + → push /
G	[+, /]	A B C - D E - * F + G	Operand→output
*	[+, *]	A B C - D E - * F + G /	* == / → pop /, push *
(	[+, *, (]	A B C - D E - * F + G /	Push (
H	[+, *, (]	A B C - D E - * F + G / H	Operand→output
-	[+, *, (, -]	A B C - D E - * F + G / H	Push -
J	[+, *, (, -]	A B C - D E - * F + G / H J	Operand→output
)	[+, *]	A B C - D E - * F + G / H J -	Pop until (
END	[]	A B C - D E - * F + G / H J - * +	Pop * then +

✓ **POSTFIX RESULT: A B C - D E - * F + G / H J - * +**

1.4 Postfix Expression Evaluation

Scan left to right. **Operand** → **push**. **Operator** → **pop 2 operands** (right = first pop, left = second pop), compute, push result.

Example: Evaluate 6 2 3 + - 3 8 2 / + * 2 ^

Token	Action	Stack
6	Push 6	[6]
2	Push 2	[6, 2]
3	Push 3	[6, 2, 3]
+	Pop 3, 2 → 2+3=5 → Push 5	[6, 5]
-	Pop 5, 6 → 6-5=1 → Push 1	[1]
3	Push 3	[1, 3]
8	Push 8	[1, 3, 8]
2	Push 2	[1, 3, 8, 2]
/	Pop 2, 8 → 8/2=4 → Push 4	[1, 3, 4]
+	Pop 4, 3 → 3+4=7 → Push 7	[1, 7]
*	Pop 7, 1 → 1*7=7 → Push 7	[7]
2	Push 2	[7, 2]

^	Pop 2,7 → $7^2=49$ → Push 49	[49]
END	Result = top	Answer: 49

1.5 Balanced Brackets / Parenthesis Checker

Scan left to right. Push opening brackets. On closing bracket: if stack empty → UNBALANCED; else pop and check match. After full scan: empty stack = BALANCED.

Expression	Trace	Result
Push{[(())]} Push [, push (, see) → pop (✓, see] → pop [✓, see } → pop { ✓, see } → pop { }		BALANCED ✓
{[(())]}	Push {, push [, push (, see] → top is (→ MISMATCH	UNBALANCED ✗
((()))	Push (, (, (, pop), pop) → stack has 1 left → not empty	UNBALANCED ✗

■ PRACTICE: STACK PRACTICE (Past Paper Pattern)

- [5 Marks] Trace: enqueue(8), display, enqueue(3), display, enqueue(10), display, dequeue, display, enqueue(1), display, dequeue, display — show queue after each step.
- [10 Marks] Convert to postfix step by step (show stack): $(A+B)*(C-D)/(E+F)-G*H$
- [5 Marks] Evaluate postfix: $5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$
- [5 Marks] Check if balanced: (a) {[()]} (b) [(I)] (c) {[()()]} Show stack trace.

■ SECTION 2: QUEUE

2.1 Linear Queue

FIFO Principle

First In, First Out — elements added at rear, removed from front. Like a real queue/line at a counter.

Enqueue(x)	Add at rear. Check overflow (size==MAX). queue[rear]=x; rear++; size++
Dequeue()	Remove from front. Check underflow (size==0). temp=queue[front]; front++; size--; return temp
Display	for(i=front; i<front+size; i++) print queue[i]
Overflow	size == MAX → cannot enqueue
Underflow	size == 0 → cannot dequeue

2.2 Sorted Enqueue — MID-TERM Past Paper Code

This code inserts in **ascending sorted order** by shifting larger elements right:

```
void enqueue(int number) {  
    int i;  
    for (i = size-1; i >= 0 && queue[i] > number; i--)  
        queue[i+1] = queue[i]; // shift elements right  
    queue[i+1] = number; // place at correct position  
    size++;  
}
```

Full Trace — Operations: enqueue(8), display, enqueue(3), display, enqueue(10), display, dequeue, display, enqueue(1), display, dequeue, display

Operation	What Happens	Queue Contents
enqueue(8)	Empty queue → insert 8 at index 0	[8]
display	Print size=1 elements	8
enqueue(3)	3 < 8 → shift 8 right: queue[1]=8, queue[0]=3	[3, 8]
display	Print	3 8
enqueue(10)	10 > 8 → no shift needed, queue[2]=10	[3, 8, 10]
display	Print	3 8 10
dequeue()	Remove front(3), shift all left: [8,10]	[8, 10]
display	Print	8 10
enqueue(1)	1<8→shift 8,10 right: queue[0]=1	[1, 8, 10]
display	Print	1 8 10
dequeue()	Remove front(1), shift left: [8,10]	[8, 10]
display	Print	8 10

2.3 Circular Queue

Solves the 'false overflow' problem. Uses **modulo arithmetic** to wrap around.

- rear = (rear + 1) % MAX on enqueue
- front = (front + 1) % MAX on dequeue
- **Full:** (rear+1)%MAX == front | **Empty:** front == rear (or use size counter)

Circular Queue Trace — MAX=5, show using **Simple Queue** and **Circular Queue** (array indices) notation:

Operation	front	rear	Array [0..4]	Simple Q
Init (empty)	0	0	[_,_,_,_,_]	empty
EnqR(10)	0	1	[10,_,_,_,_]	10
EnqR(20)	0	2	[10,20,_,_,_]	10→20
DeqF()	1	2	[_,20,_,_,_]	20
EnqR(30)	1	3	[_,20,30,_,_]	20→30
EnqR(40)	1	4	[_,20,30,40,_]	20→30→40
EnqR(50)	1	0	[50,20,30,40,_] wrap!	20→30→40→50
DeqF()	2	0	[50,_,30,40,_]	30→40→50

2.4 Double-Ended Queue (Deque) — Past Paper CLO1 C [10 Marks]

EnqueueRear(x)	rear=(rear+1)%MAX; arr[rear]=x; size++
EnqueueFront(x)	front=(front-1+MAX)%MAX; arr[front]=x; size++
DequeFront()	temp=arr[front]; front=(front+1)%MAX; size--
DequeRear()	temp=arr[rear]; rear=(rear-1+MAX)%MAX; size--

Past Paper Trace — Size 6 (indices 0–5):

Sequence: EnqR(10), EnqR(20), EnqF(5), EnqR(30), DeqF(), EnqR(10), EnqR(50), EnqF(2), DeqR(), EnqF(1)

Operation	front	rear	Array [0..5]	Contents F→R
EnqR(10)	0	0	[10,_,_,_,_,_]	10
EnqR(20)	0	1	[10,20,_,_,_,_]	10→20
EnqF(5)	5	1	[10,20,_,_,_,5]	5→10→20
EnqR(30)	5	2	[10,20,30,_,_,5]	5→10→20→30
DeqF()	0	2	[10,20,30,_,_,_]	10→20→30
EnqR(10)	0	3	[10,20,30,10,_,_]	10→20→30→10
EnqR(50)	0	4	[10,20,30,10,50,_]	10→20→30→10→50
EnqF(2)	5	4	[10,20,30,10,50,2]	2→10→20→30→10→50
DeqR()	5	3	[10,20,30,10,_,2]	2→10→20→30→10
EnqF(1)	4	3	[10,20,30,10,1,2]	1→2→10→20→30→10

■ PRACTICE: QUEUE PRACTICE

■ [5 Marks] Sorted Queue (MAX=5): enqueue(15), display, enqueue(5), display, enqueue(20), display, dequeue, display, enqueue(10), display — trace each step.

■ [10 Marks] Circular Queue (MAX=5): Enq(A),Enq(B),Enq(C),Deq,Deq,Enq(D),Enq(E),Enq(F). Show front/rear/array after each op.

■ [10 Marks] Deque (size 6):

EnqR(8),EnqR(16),EnqF(4),EnqR(24),DeqF(),EnqR(12),EnqR(32),EnqF(2),DeqR(),EnqF(1). Show front/rear/contents.

■ SECTION 3: LINKED LIST

3.1 Singly Linked List — Basics

Node Structure

Each node: [data | next →]. Last node's next = NULL. Head points to first node.

Insert at Head	newNode.next = head; head = newNode;
Insert at Tail	Traverse to last; last.next = newNode; newNode.next = NULL
Delete Head	head = head.next;
Delete Value	Find node; prev.next = curr.next; free(curr);
Traverse	curr = head; while(curr!=NULL){ print curr.data; curr=curr.next; }

3.2 ■ QUIZ 2 QUESTION A — Reverse a Linked List

■ QUIZ PATTERN: Quiz 2 Spring 2026 — Q1a

What will be the output of the following code for the list A->B->C->D->E->F?

```
ListNode* solve(ListNode* head) {
    ListNode* p = NULL;
    if(head == NULL) { return head; }
    if(head->next == NULL) { return head; }
    ListNode* curr = head->next;
    while(head != NULL) {
        head->next = p;
        p = head;
        head = curr;
        if(curr != NULL) { curr = curr->next; }
    }
    return p;
}
```

What this code does: It **reverses** the linked list. Let's trace it step by step:

Initial State: head→A→B→C→D→E→F→NULL, p=NULL, curr = B

Iteration	head	curr	p	Action
Start	A	B	NULL	Setup: curr=B
Iter 1	A	B	NULL	A.next=NULL, p=A, head=B, curr=C
Iter 2	B	C	A	B.next=A, p=B, head=C, curr=D
Iter 3	C	D	B→A	C.next=B, p=C, head=D, curr=E
Iter 4	D	E	C→B→A	D.next=C, p=D, head=E, curr=F
Iter 5	E	F	D→C→B→A	E.next=D, p=E, head=F, curr=NULL
Iter 6	F	NULL	E→D→C→B→A	F.next=E, p=F, head=NULL, curr=NULL

Exit loop	NULL	—	F→E→D→C→B→A→NULL	head==NULL → exit
Return	p	—	—	return p (new head = F)

✓ **OUTPUT:** F -> E -> D -> C -> B -> A -> NULL (List is REVERSED)

■ **KEY INSIGHT:** 3 pointers needed to reverse: p (previous), head (current), curr (next). Each node's next pointer is flipped to point backward.

3.3 ■ QUIZ 2 QUESTION B — Remove Duplicates from Sorted List

■ QUIZ PATTERN: Quiz 2 Spring 2026 — Q1b

What will be the output of the following code for the list 1->2->2->2->3->3->4->5->6?

```
void abc(struct Node* head) {
    struct Node* current = head;
    while (current != NULL && current->next != NULL) {
        if (current->data == current->next->data) {
            struct Node* temp = current->next;
            current->next = temp->next;
            free(temp);
        } else {
            current = current->next;
        }
    }
}
```

What this code does: Removes duplicate consecutive elements from a sorted linked list.

Input List: 1→2→2→2→3→3→4→5→6

Step	current	current->next	Match?	Action	List State
1	1	2	No	Move current	1→2→2→2→3→3→4→5→6
2	2	2	YES	Delete 2nd '2'	1→2→2→3→3→4→5→6
3	2	2	YES	Delete 2nd '2'	1→2→3→3→4→5→6
4	2	3	No	Move current	1→2→3→3→4→5→6
5	3	3	YES	Delete 2nd '3'	1→2→3→4→5→6
6	3	4	No	Move current	1→2→3→4→5→6
7	4	5	No	Move current	1→2→3→4→5→6
8	5	6	No	Move current	1→2→3→4→5→6
9	6	NULL	—	Loop ends	1→2→3→4→5→6

✓ **OUTPUT:** 1 -> 2 -> 3 -> 4 -> 5 -> 6 (All duplicates removed)

■ **KEY INSIGHT:** When duplicate found → do NOT move current. Delete next node and re-check (current might still match new next). When no match → advance current.

3.4 Merge Two Sorted Linked Lists — Past Paper Q2A [10 Marks]

Algorithm (step-by-step):

- Create a dummy head node (simplifies edge cases).

- Use 'tail' pointer starting at dummy.
- While both lists non-NULL: compare heads, attach smaller to tail.next, advance that list and tail.
- Append remaining non-NULL list to tail.next.
- Return dummy.next.

Trace: list1 = 10→30→50, list2 = 20→40→60→80

Step	L1 ptr	L2 ptr	Compare	Attach	Merged so far
1	10	20	10<20	10	10
2	30	20	30>20	20	10→20
3	30	40	30<40	30	10→20→30
4	50	40	50>40	40	10→20→30→40
5	50	60	50<60	50	10→20→30→40→50
6	NULL	60	L1 done	append L2	10→20→30→40→50→60→80

3.5 Linked List Rotation — Past Paper Q2B [10 Marks]

The function rotates the list left by k positions (the k-th node becomes the new head):

```
k = k % length; // handle k > length
// Make circular, find new tail at position (length - k), break circle
```

List [1→2→3→4→5→6→7], length=7:

k value	k % 7	New head (position)	Result
k=4	4	Node at pos 4+1=5 (value 5)	5→6→7→1→2→3→4
k=8	8%7=1	Node at pos 1+1=2 (value 2)	2→3→4→5→6→7→1
k=7	7%7=0	k==0 → return original head	1→2→3→4→5→6→7

3.6 Doubly Linked List & Circular Linked List

Doubly LL node	[prev ← data → next]. Allows bidirectional traversal.
DLL Insert at head	newNode.next=head; head.prev=newNode; newNode.prev=NULL; head=newNode
DLL Delete node	node.prev.next=node.next; node.next.prev=node.prev; free(node)
Circular SLL	last.next = head (not NULL). Stop condition: curr.next == head
Circular DLL	Both ends connected. last.next=head; head.prev=last

■ PRACTICE: LINKED LIST PRACTICE (QUIZ 2 Style + Past Paper)

- [5 Marks] QUIZ STYLE: Trace the reverse function on list: 1->2->3->4->5. Show p, head, curr at each iteration.
- [5 Marks] QUIZ STYLE: Trace remove-duplicates on list: 1->1->2->3->3->3->4->5->5. Show list after each deletion.
- [10 Marks] Merge two sorted lists: L1=2→5→8→11, L2=1→4→7→10→13. Show each merge step.
- [10 Marks] Rotate [1-2-3-4-5-6-7-8] for k=6 and k=10. Show length calc, new head, final list.
- [5 Marks] In DLL 1↔2↔3↔4↔5, delete node 3. Show prev/next updates.

■ SECTION 4: BINARY SEARCH TREE (BST)

4.1 BST Rules

- **Left subtree:** all values < node's value
- **Right subtree:** all values > node's value
- Both left and right subtrees are also valid BSTs
- No duplicates

4.2 Operations

Insert	Compare with root: go left if smaller, right if larger. Insert at NULL.
Search	Same path as insert. Return true if found, false if NULL.
In-order	Left → Node → Right. Gives sorted ascending output.
Pre-order	Node → Left → Right. Used to copy/serialize tree.
Post-order	Left → Right → Node. Used to delete tree or evaluate expression trees.

4.3 Delete (3 Cases) — Very Important!

Case	Condition	Action
Leaf node	No children	Simply remove (set parent pointer to NULL)
1 child Has	only left OR right child	Replace node with its single child
2 children	Has both left and right	Find in-order SUCCESSOR (leftmost of right subtree) Replace node's value with successor's value Delete successor from its original position

4.4 Past Paper BST — Insert & Delete [3+5 Marks]

Insert in order: 50, 30, 70, 20, 40, 60, 80, 35, 45, 75, 85

```
50
/ \
30 70
/ \ / \
20 40 60 80
/ \ \ \
35 45 75 85
```

Delete 70 (has 2 children): Find in-order successor = leftmost of right subtree of 70 = go right to 80, then left → **75**

- Replace 70's value with 75
- Delete node 75 from its original position (it has no left child → 1-child case)

```
50
/ \
30 75 ← replaced 70
/ \ / \
20 40 60 80
/ \ \ \
35 45 85
```

✓ **Traversals of original BST: In-order: 20,30,35,40,45,50,60,70,75,80,85 | Pre-order: 50,30,20,40,35,45,70,60,80,75,85**

■ PRACTICE: BST PRACTICE

- [3 Marks] Insert 40,20,60,10,30,50,70,25,35,65,75 into empty BST. Draw tree.
- [5 Marks] Delete node 60 from above BST. Show in-order successor and final tree.
- [3 Marks] Write in-order, pre-order, post-order traversals of the BST from Q1.
- [5 Marks] Given BST with values [45,30,60,15,35,55,70]. Delete 30. Show all steps.

SECTION 5: AVL TREE

5.1 Core Concepts

Balance Factor	$BF(\text{node}) = \text{Height}(\text{Left}) - \text{Height}(\text{Right})$. Must be -1 , 0 , or $+1$ for every node.
Height Rule	$H(\text{node}) = 1 + \max(H(\text{left}), H(\text{right}))$. $H(\text{NULL}) = -1$ (using -1 convention)

BF = 0	Equal heights — perfectly balanced
BF = +1	Left is 1 taller — still valid
BF = -1	Right is 1 taller — still valid
BF = +2	LEFT HEAVY → needs rotation
BF = -2	RIGHT HEAVY → needs rotation

5.2 Four Rotation Types — MASTER THIS TABLE

Type	Trigger	Steps	Memory Tip
LL (Right Rotate)	BF=+2 at Z BF=+1 at left child Y	1. Y becomes root 2. Z becomes Y's right child 3. Y's old right child → Z's left child	Left-Left imbalance → Single Right Rotate
RR (Left Rotate)	BF=-2 at Z BF=-1 at right child Y	1. Y becomes root 2. Z becomes Y's left child 3. Y's old left child → Z's right child	Right-Right imbalance → Single Left Rotate
LR (Double)	BF=+2 at Z BF=-1 at left child Y	1. Left Rotate Y (fix Y first) 2. Right Rotate Z	Left-Right: double rotation First left, then right
RL (Double)	BF=-2 at Z BF=+1 at right child Y	1. Right Rotate Y (fix Y first) 2. Left Rotate Z	Right-Left: double rotation First right, then left

5.3 Full AVL Trace — Past Paper [7 Marks]: Insert [50,30,70,20,40,35,80]

Insert	Tree (simplified)	Heights	BF values	Rotation?
50	50	$H(50)=0$	$BF(50)=0$	None
30	50 ■ L:30	$H(50)=1, H(30)=0$	$BF(50)=+1, BF(30)=0$	None
70	50 30 70	$H(50)=1, H(30)=0, H(70)=0$	$BF(50)=0, BF(30)=0, BF(70)=0$	None
20	50 30 70 20	$H(50)=2, H(30)=1, H(20)=0$	$BF(50)=+1, BF(30)=+1$	None
40	50 30 70 20 40	$H(30)=1$	$BF(50)=+1, BF(30)=0$	None
35	35 goes left of 40. $BF(40)=+1, BF(30)=-1$ Check 50: Left subtree height=3, Right=1 → $BF=+2$ Left child 30 has $BF=-1$ → LR ROTATION at 50!	$H(50)=3$	$BF(50)=+2$ ■	LR at 50: 1) Left rotate 30 2) Right rotate 50 New root of subtree: 40

80	Tree now rooted at 40. 40 with 30 left, 50 right Insert 80 as right of 70 All BFs check out valid	All heights balanced	All BF in {-1,0,+1}	None
----	--	----------------------	---------------------	------

Final AVL Tree after LR rotation at 50, then inserting 80:

40
/ \
30 50
/ \ / \
20 35 45 70
\
80

■ EXAM TIP: After EVERY insertion, go bottom-up and recalculate BFs. Fix the LOWEST unbalanced node first. LR/RL need 2 rotations.

5.4 AVL Second Trace — Insert [10,20,30,40,50] (Pure RR rotations)

Insert	Before Rotation	BF Issue	After Rotation
10	10	BF(10)=0	10 (root)
20	10→R:20	BF(10)=-1	OK
30	10→R:20→R:30	BF(10)=-2 (Left-Right imbalance at 10): 20 is new root, 10→left, 30→right	OK
40	20 with 10L, 30R; 30→R:40	BF(20)=-1	OK
50	30→R:40→R:50	BF(20)=-2 (Left-Right imbalance at 20): 40 is new root of that subtree	OK

■ PRACTICE: AVL TREE PRACTICE

- [7 Marks] Insert [50,30,70,20,40,35,80] step by step. Show tree+BF after each insertion. Perform rotations.
- [5 Marks] Insert [10,20,30,40,50] — multiple RR rotations needed. Show each.
- [7 Marks] Insert [30,20,40,10,25,35,50,5,15,28,27]. Identify every rotation type.
- [5 Marks] Insert [3,2,1,4,5,6,7] into AVL. Show all rotations.

SECTION 6: HEAP TREE, HEAP SORT & PRIORITY QUEUE

6.1 Heap Concepts

Max Heap	Every parent $\geq$ both children. Root = Maximum element. Used for descending sort.
----------	--

Min Heap	Every parent $\leq$ both children. Root = Minimum element. Used for ascending sort.
----------	---

Array Representation (0-indexed):

- Parent of node i : $(i-1)/2$
- Left child of i : $2i+1$
- Right child of i : $2i+2$
- Last non-leaf node (for heapify): $(n/2)-1$

6.2 Heap Insertion (Bubble-Up / Sift-Up)

- Add new element at the end of the array.
- Compare with parent: if violates heap property $\rightarrow$ swap.
- Repeat upward until property satisfied or root reached.

Insert 35 into Max-Heap [80,70,60,50,40,30,20]:

Step	Action	Array State
1	Add 35 at end (index 7)	[80, 70, 60, 50, 40, 30, 20, 35]
Parent of idx7 = $(7-1)/2=3 \rightarrow$ value 50. $35 < 50 \rightarrow$ no swap		DONE
Result	Heap valid	[80, 70, 60, 50, 40, 30, 20, 35] ✓

Insert 90 into same heap [80,70,60,50,40,30,20]:

Step	Action	Array
1	Add 90 at index 7	[80, 70, 60, 50, 40, 30, 20, 90]
2	Parent of 7 = idx3 = 50. $90 > 50 \rightarrow$ SWAP	[80, 70, 60, 90, 40, 30, 20, 50]
3	Parent of 3 = idx1 = 70. $90 > 70 \rightarrow$ SWAP	[80, 90, 60, 70, 40, 30, 20, 50]
4	Parent of 1 = idx0 = 80. $90 > 80 \rightarrow$ SWAP	[90, 80, 60, 70, 40, 30, 20, 50]
5	At root – DONE	[90, 80, 60, 70, 40, 30, 20, 50] ✓

6.3 Heap Deletion — Extract Root (Max/Min)

- Replace root with the **last element** of the array.
- Reduce heap size by 1 (remove last position).
- 'Heapify-down' (Sift-Down): Compare new root with children, swap with the **larger child** (Max-Heap) until valid.

Delete root from Max-Heap [80,70,60,50,40,30,20]:

Step	Action	Array
1	Move last (20) to root, remove 80	[20, 70, 60, 50, 40, 30]
Compare 20 with children 70, 60. Max child=70. $20 < 70 \rightarrow$ swap idx0,1		[70, 20, 60, 50, 40, 30]
Compare 20 (now at idx1) with children 50, 40. Max=50. $20 < 50 \rightarrow$ swap		[70, 50, 60, 20, 40, 30]
Compare 20 (at idx3) with children – none in range $\rightarrow$ DONE		[70, 50, 60, 20, 40, 30] ✓

6.4 Heap Sort — Complete Algorithm [Most Important!]

Phase 1: Build Max-Heap — Start from last non-leaf ($n/2-1$) and heapify-down each node.

Phase 2: Extract — Swap root (max) with last element, reduce size by 1, heapify root. Repeat.

Sort [4, 10, 3, 5, 1] step by step:

Phase 1 — Build Max-Heap: Last non-leaf = idx 1 (value 10)

Step	Action	Array
Heapify children of idx1: idx3=5, idx4=1. $10 > \text{both}$ → no swap		[4, 10, 3, 5, 1]
Heapify child of idx0: idx1=10, idx2=3. $\text{Max}=10$. $4 < 10$ → swap		[10, 4, 3, 5, 1]
Continue idx1 Children: idx3=5, idx4=1. $4 < 5$ → swap		[10, 5, 3, 4, 1]
Max-Heap ready		[10, 5, 3, 4, 1] ✓

Phase 2 — Extract Max repeatedly:

Extract	Swap root with	After swap + heapify	Sorted portion
Extract 10	last(1) → [1, 5, 3, 4, 10]	Heapify [1, 5, 3, 4]: [5, 4, 3, 1]	[..., 10]
Extract 5	last(1) → [1, 4, 3, 5]	Heapify [1, 4, 3]: [4, 1, 3]	[..., 5, 10]
Extract 4	last(3) → [3, 1, 4]	Heapify [3, 1]: [3, 1]	[..., 4, 5, 10]
Extract 3	last(1) → [1, 3]	Size=1 done	[1, 3, 4, 5, 10] ✓

✓ **SORTED RESULT (ascending): [1, 3, 4, 5, 10]**

Time: $O(n \log n)$ — best, avg, worst. **Space:** $O(1)$ in-place. **Not stable.**

6.5 Priority Queue using Heap

ADT where elements are served by priority, not arrival order.

Max Priority Q	Max-Heap. Highest value served first. Enqueue= $O(\log n)$, Dequeue= $O(\log n)$
Min Priority Q	Min-Heap. Smallest value = highest priority.
Real-world use	OS scheduling, Dijkstra's algorithm, A* pathfinding, Huffman encoding, hospital triage
enqueue(x)	Insert x at end, bubble-up to restore heap property.
dequeue()	Remove root (top priority), replace with last, heapify-down.

■ PRACTICE: HEAP PRACTICE

- [5 Marks] Build a Max-Heap by inserting: 3,1,6,5,2,4 one at a time. Show bubble-up at each step.
- [5 Marks] Delete root from Max-Heap [90,80,70,60,50,40,30]. Show full heapify-down trace.
- [7 Marks] Sort [12,11,13,5,6,7] using Heap Sort. Show Phase 1 (build) and Phase 2 (extract) fully.
- [3 Marks] Build Min-Heap from [15,10,12,8,3,7,5]. What is extracted first? Second? Third?

■ SECTION 7: PRACTICE EXAMINATION PAPERS

Mock Quiz 2 — Spring 2026 Pattern (10 Marks, 20 minutes)

Course: Data Structures (CSC211) | Instructor: Ms. Humera Niaz | Section A | SP25-BSE

Question 1: [Employ linear data structures to solve computing problems]

Part a) [5 Marks]

What will be the output of the following code for the list P->Q->R->S->T?

```
ListNode* solve(ListNode* head) {  
    ListNode* p = NULL;  
    if(head == NULL) { return head; }  
    if(head->next == NULL) { return head; }  
    ListNode* curr = head->next;  
    while(head != NULL) {  
        head->next = p;  
        p = head;  
        head = curr;  
        if(curr != NULL) { curr = curr->next; }  
    }  
    return p;  
}
```

Answer space — trace p, head, curr at each iteration, state final list:

Part b) [5 Marks]

What will be the output of the following code for the list: 2->3->3->4->4->4->5->6->6?

```
void abc(struct Node* head) {  
    struct Node* current = head;  
    while (current != NULL && current->next != NULL) {  
        if (current->data == current->next->data) {  
            struct Node* temp = current->next;  
            current->next = temp->next;  
            free(temp);  
        } else {  
            current = current->next;  
        }  
    }  
}
```

Answer space — show list state after each change:

Mock Mid-Term Exam — Fall/Spring 2026 Pattern (60 Marks, 1h 30min)

All questions compulsory. Show all work. Plagiarism = F grade.

Question 1 (25 Marks)

[Employ linear data structures to solve computing problems]

Part A [5 Marks]

Trace the sorted enqueue code (ascending order, MAX=5) for the operations:

enqueue(12), display, enqueue(4), display, enqueue(18), display, dequeue(), display, enqueue(7), display, dequeue(), display

After each operation, state the current contents of the queue.

Part B [10 Marks]

Convert the following infix expression to postfix using a stack. Show stack state and output after EVERY token:

$(A-B)*(C+D)/(E-F)+G*H$

Part C [10 Marks]

Perform the following operations on a Deque (array size 6, indices 0–5). Show front, rear, array contents, and queue contents after EACH operation:

EnqueueRear(5), EnqueueRear(10), EnqueueFront(3), EnqueueRear(15), DequeueFront(), EnqueueRear(12), EnqueueRear(20), EnqueueFront(1), DequeueRear(), EnqueueFront(0)

Question 2 (20 Marks)

[Employ linear data structures to solve computing problems]

Part A [10 Marks]

Merge the following two sorted linked lists. Write a stepwise algorithm and trace each comparison step:

list1 = 1 → 4 → 7 → 10

list2 = 2 → 3 → 8 → 11 → 15

Show the merged list after each attachment step.

Part B [10 Marks]

Apply the LinkedList rotation function on the singly linked list [1–2–3–4–5–6–7–8] with head at node 1. What would be the output for $k = 3$ and $k = 11$?

Show: length calculation, $k \% \text{length}$, position of new head, final output list.

Question 3 (15 Marks)

[Use non-linear data structures to solve computing problems]

Part A [3 Marks]

Insert the following values into an initially empty BST: 45, 25, 65, 15, 35, 55, 75, 10, 20, 40, 50. Draw the resulting BST.

Part B [5 Marks]

From the BST in Part A, delete node 65. Show: (a) in-order successor identification (b) updated tree structure with all pointer changes.

Part C [7 Marks]

Insert [15, 10, 20, 5, 12, 17, 25, 3, 7] into an initially empty AVL Tree. Show the tree and BF of each node after each insertion. Perform rotations whenever required — clearly indicate rotation type.

■ QUICK REFERENCE — EXAM CHEAT SHEET

Key Formulas & Rules

Concept	Formula / Key Fact
Stack LIFO	push: <code>arr[top++]=x</code> pop: <code>return arr[--top]</code> peek: <code>arr[top-1]</code>
Queue FIFO	enqueue at rear, dequeue from front Circular: $(i+1)\%MAX$
BF formula	$BF = H(\text{left subtree}) - H(\text{right subtree})$ Valid: $\{-1, 0, +1\}$
Height	$H(\text{node}) = 1 + \max(H(\text{left}), H(\text{right}))$ $H(NULL) = -1$
Heap array	Parent: $(i-1)/2$ Left child: $2i+1$ Right child: $2i+2$
Heap Sort	$O(n \log n)$ time, $O(1)$ space, in-place, NOT stable
Infix→Postfix	$\wedge$ prec 3 > */ prec 2 > +- prec 1 '(' push, ')' pop till '('
Postfix eval	Operand→push Operator→pop 2, compute (left op second pop), push result
BST Delete 2-child	Replace with in-order SUCCESSOR (leftmost node of RIGHT subtree)
AVL LL	$BF=+2$ at Z, $BF=+1$ at Y → Single RIGHT rotate at Z
AVL RR	$BF=-2$ at Z, $BF=-1$ at Y → Single LEFT rotate at Z
AVL LR	$BF=+2$ at Z, $BF=-1$ at Y → LEFT rotate Y, then RIGHT rotate Z
AVL RL	$BF=-2$ at Z, $BF=+1$ at Y → RIGHT rotate Y, then LEFT rotate Z
Reverse LL	3 pointers: <code>prev=NULL, curr=head, next</code> . Each step: <code>curr.next=prev; prev=curr; curr=next</code>
Remove Dups	Sorted LL: if <code>curr.data==curr.next.data</code> → delete <code>curr.next</code> , don't advance <code>curr</code>
Merge sorted LL	Compare heads, attach smaller, advance that pointer, append remaining at end
LL Rotate k	Calculate len; make circular; move $(len-k)$ steps from head; <code>newHead=curr.next; curr.next=NULL</code>

Common Mistakes to Avoid

Topic	■ Common Mistake	✓ Correct Approach
Infix→Postfix	Popping when stack empty	Always check <code>isEmpty()</code> before popping
Postfix Eval	Wrong operand order	Second pop = LEFT operand, first pop = RIGHT
BST Delete	Deleting without finding successor	Should delete: ALWAYS use in-order successor
AVL Rotation	Wrong rotation for LR case	$LR \neq LR$ rotate! It's LEFT then RIGHT (double)
BF Calculation	Counting nodes not height	BF uses HEIGHT not count. $H(null)=-1$
Heap Deletion	Just removing root	Move LAST element to root, THEN heapify down
Reverse LL	Losing next pointer	Save <code>curr.next</code> BEFORE changing <code>curr.next</code>
Remove Dups	Advancing when duplicate found	When dup found: DELETE next, do NOT advance

■ You've got this! Cover each section, practice the traces, and you'll ace it! Remember:
Show ALL your work step by step — partial credit counts!